

Triển khai mô hình SVM Iris

FastAPI và Render

- Xây dựng mô hình Support Vector Machine (SVM) cho bộ dữ liệu Iris.
- Lưu mô hình đã huấn luyện bằng `joblib`.
- Đóng gói mô hình thành REST API bằng FastAPI.
- Kiểm thử API trên máy cục bộ và qua Swagger UI.
- Đưa mã nguồn lên GitHub và triển khai trên Render.

Client → FastAPI → SVM model → Prediction

Cấu trúc thư mục dự án

```
iris-fastapi/  
|-- train.py  
|-- app.py  
|-- svm_model.pkl  
|-- requirements.txt  
|-- render.yaml  
`-- Procfile
```

Lưu ý

File `svm_model.pkl` được tạo sau khi chạy `train.py`.

Bước 1: Huấn luyện mô hình SVM

Tạo file train.py:

```
from sklearn import datasets
from sklearn.svm import SVC
import joblib

iris = datasets.load_iris()
X = iris.data
y = iris.target

model = SVC(kernel="linear")
model.fit(X, y)

joblib.dump(model, "svm_model.pkl")
print("Model saved!")
```

Bước 1: Cài thư viện và chạy huấn luyện

Cài đặt các thư viện cần thiết:

```
python -m pip install scikit-learn joblib
```

Chạy chương trình huấn luyện:

```
python train.py
```

Kết quả mong đợi:

```
Model saved!
```

Trong thư mục dự án xuất hiện file `svm_model.pkl`.

Bước 2: Khởi tạo ứng dụng FastAPI

Tạo phần đầu của file `app.py`:

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

model = joblib.load("svm_model.pkl")

app = FastAPI(
    title="Iris Classification API",
    description="SVM model for the Iris dataset",
    version="1.0.0",
)

class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float
```

Bước 2: Tạo các endpoint

Viết tiếp trong file `app.py`:

```
species = {
    0: "setosa",
    1: "versicolor",
    2: "virginica",
}

@app.get("/")
def home():
    return {"message": "Iris SVM API is running"}

@app.get("/health")
def health():
    return {"status": "healthy"}
```

Bước 2: Endpoint dự đoán

Viết tiếp trong file `app.py`:

```
@app.post("/predict")
def predict(data: IrisInput):
    features = [
        data.sepal_length,
        data.sepal_width,
        data.petal_length,
        data.petal_width,
    ]

    prediction = int(model.predict(features)[0])

    return {
        "class_id": prediction,
        "prediction": species[prediction],
    }
```

Bước 3: Chạy thử trên máy cục bộ

Cài FastAPI và Uvicorn:

```
python -m pip install fastapi uvicorn
```

Khởi động server ở chế độ phát triển:

```
uvicorn app:app --reload
```

Các địa chỉ kiểm thử:

- Endpoint gốc: `http://127.0.0.1:8000/`
- Swagger UI: `http://127.0.0.1:8000/docs`
- Health check: `http://127.0.0.1:8000/health`

Bước 4: Kiểm thử API bằng Swagger UI

Dữ liệu gửi đến endpoint POST /predict:

```
{  
  "sepal_length": 5.1,  
  "sepal_width": 3.5,  
  "petal_length": 1.4,  
  "petal_width": 0.2  
}
```

Kết quả dự kiến:

```
{  
  "class_id": 0,  
  "prediction": "setosa"  
}
```

Bước 5: Tạo requirements.txt

Tạo file requirements.txt:

```
fastapi
uvicorn[standard]
scikit-learn
joblib
pydantic
```

Có thể kiểm tra việc cài đặt bằng:

```
python -m pip install -r requirements.txt
```

Khuyến nghị

Khi cần tái lập môi trường chính xác, hãy khóa phiên bản thư viện sau khi đã kiểm thử thành công.

Bước 6: Tạo Procfile

Tạo file có tên chính xác là `Procfile`, không có phần mở rộng:

```
web: uvicorn app:app --host 0.0.0.0 --port $PORT
```

Trong đó:

- `app` thứ nhất là tên file `app.py`.
- `app` thứ hai là biến `app = FastAPI(...)`.
- `$PORT` là cổng do Render cung cấp khi chạy dịch vụ.

Bước 7: Tạo render.yaml

Tạo file `render.yaml`:

```
services:
  - type: web
    name: iris-svm-api
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /health
```

File này mô tả dịch vụ web và cho phép Render đọc cấu hình triển khai từ repository.

Bước 8: Đưa mã nguồn lên GitHub

Khởi tạo repository cục bộ và tạo commit:

```
git init
git add .
git commit -m "Deploy Iris SVM API"
```

Kết nối repository GitHub và đẩy nhánh main:

```
git branch -M main
git remote add origin https://github.com/USERNAME/iris-fastapi.git
git push -u origin main
```

Thay USERNAME bằng tên tài khoản GitHub của bạn.

Bước 9: Tạo Web Service trên Render

- 1 Đăng nhập Render và kết nối tài khoản GitHub.
- 2 Chọn **New** rồi chọn **Web Service**.
- 3 Chọn repository `iris-fastapi`.
- 4 Chọn nhánh `main`.
- 5 Chọn Python làm runtime.
- 6 Chọn gói dịch vụ phù hợp với nhu cầu.
- 7 Nhấn **Create Web Service**.

Nếu repository chứa `render.yaml`, cũng có thể tạo dịch vụ dưới dạng Blueprint.

Bước 10: Cấu hình triển khai thủ công

Nếu không dùng Blueprint, nhập các giá trị sau trên Render:

Name: `iris-svm-api`

Build Command: `pip install -r requirements.txt`

Start Command: `uvicorn app:app -host 0.0.0.0 -port $PORT`

Render sẽ thực hiện tuần tự:

- 1 Clone repository.
- 2 Cài các dependency.
- 3 Khởi động Uvicorn và FastAPI.
- 4 Cung cấp URL công khai cho ứng dụng.

Bước 11: Kiểm tra sau khi deploy

Giả sử URL Render là:

```
https://iris-svm-api.onrender.com
```

Kiểm tra các địa chỉ:

```
https://iris-svm-api.onrender.com/  
https://iris-svm-api.onrender.com/health  
https://iris-svm-api.onrender.com/docs
```

Nếu ứng dụng không khởi động, xem phần log của dịch vụ để xác định lỗi cài dependency, sai tên module hoặc thiếu file mô hình.

Bước 12: Gọi API từ Python

Cài thư viện requests, sau đó chạy đoạn mã sau:

```
import requests

url = "https://iris-svm-api.onrender.com/predict"
data = {
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2,
}

response = requests.post(url, json=data, timeout=30)
response.raise_for_status()
print(response.json())
```

Bước 13: Gọi API bằng curl

Có thể kiểm thử trực tiếp từ terminal:

```
curl -X POST \  
  "https://iris-svm-api.onrender.com/predict" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "sepal_length": 5.1,  
    "sepal_width": 3.5,  
    "petal_length": 1.4,  
    "petal_width": 0.2  
  }'
```

Bước 14: Gọi API từ JavaScript

```
fetch("https://iris-svm-api.onrender.com/predict", {  
  method: "POST",  
  headers: {"Content-Type": "application/json"},  
  body: JSON.stringify({  
    sepal_length: 5.1,  
    sepal_width: 3.5,  
    petal_length: 1.4,  
    petal_width: 0.2  
  })  
})  
  .then(response => response.json())  
  .then(data => console.log(data));
```

Một số lỗi thường gặp

- **ModuleNotFoundError:** thiếu package trong `requirements.txt`.
- **FileNotFoundError:** chưa commit file `svm_model.pkl` hoặc sai đường dẫn.
- **Application failed to start:** sai lệnh `uvicorn app:app`.
- **Port binding error:** không dùng cổng do biến môi trường `PORT` cung cấp.
- **422 Unprocessable Entity:** JSON gửi lên thiếu trường hoặc sai kiểu dữ liệu.
- **Phiên bản không tương thích:** phiên bản `scikit-learn` khi chạy khác đáng kể so với lúc lưu mô hình.

Cấu trúc có thể mở rộng

Khi dự án lớn hơn, có thể tổ chức lại như sau:

```
project/  
|-- app.py  
|-- schemas.py  
|-- predict.py  
|-- model/  
|   |-- svm_model.pkl  
|-- requirements.txt  
|-- Procfile  
|-- render.yaml
```

Có thể thay SVM bằng Random Forest, XGBoost, PyTorch hoặc TensorFlow mà vẫn giữ kiến trúc API tương tự.

Quy trình triển khai gồm:

- 1 Huấn luyện và lưu mô hình SVM.
- 2 Tạo API dự đoán bằng FastAPI.
- 3 Kiểm thử cục bộ bằng Uvicorn và Swagger UI.
- 4 Khai báo dependency và lệnh khởi động.
- 5 Đưa mã nguồn lên GitHub.
- 6 Tạo Web Service trên Render.
- 7 Kiểm thử API công khai bằng Python, curl hoặc JavaScript.